ProgressProbe: Temporal-Aware Weak Supervision for Window-Level Progress Estimation in LLM Agent Trajectories

Anonymous

Abstract

LLM-based agents frequently exhibit *semantic stagnation*—executing superficially diverse but collectively ineffective action sequences during long-horizon tasks. Detecting such stagnation at the window level requires labels that are expensive to obtain manually and noisy when generated automatically via labeling functions (LFs). We observe that LFs on sequential agent trajectories violate the i.i.d. assumption underlying standard data programming: some LFs *lead* stagnation onset (e.g., command repetition fires $\sim$1.8 steps early) while others *lag* (e.g., outcome back-propagation fires $\sim$2.1 steps late). Exploiting this, we propose a **temporal label model** that weights LF votes according to their empirically estimated temporal roles, improving gold-set label quality by +6 pp macro-F1 over majority vote and +3 pp over FlyingSquid. A lightweight GRU **ProgressProbe** trained on these labels achieves 0.80 macro-F1 on a 400-window human-annotated gold set, matching or slightly exceeding the LF oracle (+2 pp) without requiring LF computation at inference. Compared to LLM-as-judge with chain-of-thought prompting, the probe attains comparable accuracy (-2 pp vs. best LLM CoT) at 700$\times$ lower latency and $\sim$850$\times$ lower cost. Cross-agent transfer from SWE-agent to OpenHands shows the probe degrades less than the LF oracle (-9 pp vs. -14 pp), suggesting it captures implementation-agnostic progress signals. Extensive ablations over encoder choice, window size, architecture, and LF contributions confirm the robustness of the approach.

1 Introduction

Large language model (LLM) agents are increasingly deployed on complex, long-horizon software engineering tasks, where a single trajectory may span dozens of tool invocations, file edits, and test executions. A recurring failure mode is *semantic stagnation*: the agent produces actions that are individually plausible and superficially diverse, yet collectively fail to advance the task goal. Unlike simple repetition loops, semantic stagnation evades cycle detectors because each action differs syntactically from its predecessors.

Existing monitoring approaches fall into several categories, none of which fully address window-level progress estimation. Cycle detectors [Chen et al., 2024] capture structural repetition but miss semantically diverse stagnation. Process reward models (PRMs) [Lightman et al., 2023, Wang et al., 2024b] score individual steps but require costly per-step outcome labels and lose temporal dynamics within windows. Self-reflection methods [Shinn et al., 2023, Yao et al., 2023] rely on the stagnating model's own judgment. LLM-as-judge approaches provide high accuracy but at latencies ($>$ 2 s/window) and costs ($0.50+/1K windows) incompatible with real-time monitoring.

We reformulate the problem as **window-level progress estimation** and propose a three-layer solution. First, we design five labeling functions capturing complementary stagnation signals (command repetition, embedding diversity drop, error patterns, tool misuse, and outcome back-propagation) and analyze their temporal dynamics relative to stagnation onset (§3.3). We find

that LFs exhibit systematic leading/lagging behavior, violating the conditional independence assumption of standard data programming. Second, we propose a **temporal label model** (§3.4) that exploits these temporal roles for improved label aggregation, outperforming majority vote by +6 pp and FlyingSquid by +3 pp on a human gold set. Third, we train a lightweight **ProgressProbe**—a 542K-parameter GRU operating on frozen embeddings—that distills weak labels into a fast inference-time classifier (§3.5), achieving 0.80 gold macro-F1 at 4 ms/window.

Our contributions are:

1. An empirical analysis of LF temporal dynamics on agent trajectories, revealing leading/lagging structure that violates i.i.d. assumptions (§3.3).
2. A temporal-aware label aggregation method that exploits domain-specific leading/lagging indicators for improved weak supervision on sequential data (§3.4).
3. A systematic empirical study including cross-implementation transfer, LLM-as-judge comparison, and extensive ablations (§4).

2 Related Work

Agent Failure Detection. Cycle detection approaches such as CDCS/CDSA [Chen et al., 2024] and SpecRA [Wang et al., 2024a] identify structural repetition in agent traces. TrajAD [Zhang et al., 2024] applies anomaly detection to trajectory embeddings. These methods target surface-level repetition and miss semantically diverse stagnation patterns.

Process Reward Models. PRMs [Lightman et al., 2023, Wang et al., 2024b] assign step-level correctness scores. While effective in mathematical reasoning, they require expensive per-step outcome labels and aggregate uniformly within windows, losing temporal dynamics. Our approach is a weak-supervision alternative that requires no step-level labels.

Agent Self-Reflection. Reflexion [Shinn et al., 2023], LATS [Zhou et al., 2023], and Tree of Thoughts [Yao et al., 2023] enable agents to critique and revise their actions. These are intervention mechanisms, not monitoring tools, and rely on the stagnating model's own judgment.

Data Programming and Weak Supervision. Snorkel [Ratner et al., 2017] introduced programmatic labeling via noisy LFs aggregated by a generative label model. FlyingSquid [Fu et al., 2020] extends this with efficient triplet-based dependency estimation. Both assume i.i.d. data points, which is violated in sequential agent trajectories. Our temporal label model explicitly addresses this gap.

LLM-as-Judge. Using LLMs to evaluate other LLM outputs [Zheng et al., 2023] achieves high accuracy but at significant latency and cost, making it impractical for real-time trajectory monitoring.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$ of T actions, we partition τ into overlapping windows of size k: $w_i = (a_i, a_{i+1}, \ldots, a_{i+k-1})$. The task is binary classification: $y_i \in \{\textsc{Progress}, \textsc{Stagnation}\}$

Table 1: LF temporal dynamics relative to stagnation onset ($n = 4{,}820$ onset windows).

LF	Median Offset	Std	Cohen's d	Role	Significant
command_repetition	-1.8	0.9	0.62	Leading	Yes
diversity_drop	-1.2	1.1	0.31	Weak lead	Marginal
tool_misuse	-0.2	1.0	0.08	Concurrent	No
error_pattern	$+0.3$	0.8	0.12	Concurrent	No
outcome_bp	$+2.1$	1.3	0.45	Lagging	Yes

Table 2: Significant LF temporal pairs.

Leading LF	Lagging LF	Δ Steps	p-value
command_repetition	outcome_bp	3.9	< 0.001
diversity_drop	outcome_bp	3.3	< 0.001

for each window w_i.

3.2 Labeling Functions

We design five LFs capturing complementary stagnation signals:

1. **Command Repetition** (λ_{cmd}): Detects repeated shell commands or API calls within a window via exact and fuzzy matching.

2. **Diversity Drop** (λ_{div}): Measures pairwise cosine similarity of action embeddings; fires when mean similarity exceeds a threshold, indicating low semantic diversity.

3. **Error Pattern** (λ_{err}): Identifies recurring error messages or failure patterns across consecutive actions.

4. **Tool Misuse** (λ_{tool}): Detects repeated invocations of tools with invalid arguments or in inappropriate contexts.

5. **Outcome Back-Propagation** (λ_{out}): Uses trajectory-final exit context (exit codes, error messages) to retrospectively label preceding windows.

3.3 LF Temporal Dynamics

We analyze the temporal offset of each LF relative to stagnation onset across 4,820 onset windows. Define the offset δ_λ of LF λ as the median number of steps between λ's first firing and the annotated stagnation onset. Negative δ indicates a *leading* indicator; positive δ indicates a *lagging* indicator.

Table 1 shows the results. Command repetition leads stagnation by 1.8 steps (Cohen's $d = 0.62$, $p < 0.001$), while outcome back-propagation lags by 2.1 steps ($d = 0.45$). Two LF pairs show statistically significant temporal separation: command_repetition $\rightarrow$ outcome_bp ($\Delta = 3.9$ steps, $p < 0.001$) and diversity_drop $\rightarrow$ outcome_bp ($\Delta = 3.3$ steps, $p < 0.001$). The remaining LFs (error_pattern, tool_misuse) show negligible temporal differentiation ($|d| < 0.15$).

Table 3: Label quality comparison on the human gold set (400 windows).

Label Model	Gold F1	Gold Acc	Conf=1 Acc	Conf≥2 Acc	Interpretability
Majority Vote	0.720	0.740	0.620	0.890	High (trivial)
FlyingSquid	0.750	0.770	0.670	0.895	Low (latent)
Temporal LM	**0.780**	**0.800**	**0.730**	**0.910**	High (explicit)

Table 4: Temporal label model ablation: contribution of temporal weighting by LF pair.

Ablation	Gold Macro-F1	Δ vs Full
Full Temporal LM	0.780	—
Remove cmd_rep–outcome_bp offset	0.755	−2.5 pp
Remove div_drop–outcome_bp offset	0.768	−1.2 pp
Remove all other offsets	0.776	−0.4 pp
Uniform weights (= MV)	0.720	−6.0 pp

3.4 Temporal Label Model

Standard majority vote treats all LF votes equally regardless of temporal context. We propose temporal-weighted voting that adjusts LF weights based on their empirically estimated temporal roles. Specifically, for each LF λ_j with estimated offset δ_j, we compute a position-dependent weight $w_j(t)$ that up-weights leading LFs early in a stagnation episode and lagging LFs later. The aggregated soft label for window w_i is:

$$\hat{y}_i = \sigma\left(\sum_j w_j(t_i)\,\lambda_j(w_i)\right)$$

where t_i is the relative position within the trajectory and σ is the sigmoid function.

Table 3 compares label quality on the human gold set ($n = 400$). The temporal label model achieves 0.78 macro-F1, improving over majority vote by +6 pp and over FlyingSquid by +3 pp. The gain concentrates in low-confidence (conf=1) windows where LFs disagree: temporal LM achieves 0.73 accuracy vs. MV's 0.62 (+11 pp). On high-confidence (conf≥2) windows where LFs agree, both methods perform similarly (0.91 vs. 0.89).

An ablation of temporal weighting contributions (Table 4) confirms that the gain is concentrated in two LF pairs: removing the command_rep–outcome_bp offset reduces F1 by 2.5 pp, and removing the diversity_drop–outcome_bp offset reduces it by 1.2 pp. All other offsets contribute only 0.4 pp combined.

3.5 ProgressProbe

ProgressProbe is a 2-layer GRU (hidden size 256) operating on frozen Qwen3-Embedding-0.6B [Qwen, 2025] representations of actions within each window. The final hidden state is passed through a linear classifier with cross-entropy loss. The probe has 542K parameters and runs in 4 ms per window at inference.

Training uses the temporal label model's soft labels on ∼15K windows (conf≥1). We use a learning rate of 5×10^{-4} with cosine annealing, batch size 64, and train for 20 epochs with early stopping on a validation split.

Table 5: Inter-annotator agreement on the gold set by category.

Category	#Windows	Fleiss' κ	Agreement %
Clear Progress	198	0.82	91%
Clear Stagnation	130	0.78	87%
Ambiguous	72	0.45	62%
Overall	400	0.68	82%

4 Experiments

4.1 Data and Evaluation Setup

Trajectory Data. We collect 54,809 windows ($k = 5$) from SWE-agent trajectories on SWE-bench Lite instances. Windows are labeled by the five LFs and aggregated using the temporal label model. The stagnation rate in auto-labeled data is 28.5%.

Human Gold Set. We construct a gold set of 400 windows annotated by 3 annotators with stratified sampling by confidence level and MV label (Table 5). Overall Fleiss' $\kappa = 0.68$ (substantial agreement). Agreement is high for clear progress ($\kappa = 0.82$) and clear stagnation ($\kappa = 0.78$), but lower for ambiguous windows ($\kappa = 0.45$, 18% of total). The gold stagnation rate is 38%, higher than the auto-labeled rate due to intentional oversampling of ambiguous cases. We report results on both the full gold set and note that $\kappa = 0.68$ is at the lower boundary of "substantial agreement"; we discuss implications in §6.

Baselines. We compare against: (1) **Repeat-3**: a rule that flags windows containing $\geq$3 repeated commands; (2) **CosineSim**: thresholding mean pairwise cosine similarity; (3) **Structural features**: logistic regression and GBDT on 8 numeric features; (4) **LF Oracle**: applying LFs directly at inference with MV or temporal LM aggregation; (5) **FlyingSquid**: correlation-aware label model; (6) **LLM-as-Judge**: Claude Sonnet 3.5 and GPT-4o in zero-shot, few-shot, and CoT settings.

4.2 Main Results

Probe vs. Baselines on Gold Set. Table 6 presents the main comparison. ProgressProbe (Temporal LM) achieves 0.80 gold macro-F1, outperforming the LF Oracle (MV) by +8 pp and the LF Oracle (Temporal LM) by +2 pp. The latter demonstrates a modest student-exceeds-teacher effect: the probe, trained on noisy temporal LM labels, slightly surpasses the label model that generated its training data, without requiring LF computation at inference.

Cost–Accuracy Tradeoff. The probe achieves comparable accuracy to the best LLM CoT variant (Claude CoT: 0.82; probe: 0.80; $\Delta = -2$ pp) while being $\sim$700$\times$ faster (4 ms vs. 3.2 s) and $\sim$850$\times$ cheaper ($0.001 vs. $0.85 per 1K windows). For real-time monitoring of agent trajectories, this cost–accuracy tradeoff strongly favors the probe.

CosineSim Baseline. The CosineSim baseline achieves only 0.31 macro-F1 at the default (median) threshold and 0.58 even with oracle threshold optimization (Table 7). The weak correlation between cosine similarity and progress labels ($r = -0.18$) confirms that a single similarity statistic

Table 6: Main results on the human gold set. ProgressProbe matches LLM CoT accuracy at 700× lower latency.

Method	Gold F1	F1-Stag	F1-Prog	Latency	Cost/1K
Structural GBDT	0.680	—	—	—	—
CosineSim (optimal)	0.583	—	—	—	—
Repeat-3	0.630	—	—	—	—
LF Oracle (MV)	0.720	—	—	req. LFs	—
LF Oracle (Temporal LM)	0.780	—	—	req. LFs	—
GPT-4o Zero-shot	0.710	0.632	0.788	2.8 s	$0.38
GPT-4o Few-shot	0.760	0.698	0.822	3.0 s	$0.48
GPT-4o CoT	0.800	0.751	0.849	3.4 s	$0.62
Claude Zero-shot	0.730	0.658	0.802	3.2 s	$0.52
Claude Few-shot	0.780	0.721	0.839	3.5 s	$0.68
Claude CoT	**0.820**	**0.775**	**0.865**	4.1 s	$0.85
Probe (MV, conf≥2)	0.770	—	—	4 ms	$0.001
Probe (Temporal LM)	0.800	0.750	0.850	4 ms	$0.001

Table 7: CosineSim threshold sensitivity analysis.

Threshold	Precision	Recall	Macro-F1
0.91 (median)	0.42	0.38	0.313
0.90	0.48	0.51	0.470
0.872 (optimal)	0.57	0.60	0.583
0.85	0.53	0.64	0.550

cannot distinguish progress from stagnation: high similarity can indicate either effective sequential actions or repetitive ineffective ones.

Non-NLP Structural Baseline. Structural features alone (lines added/deleted, file count, command count, etc.) achieve at most 0.71 macro-F1 with GBDT (Table 8), 19 pp below the probe. This confirms that progress estimation requires semantic understanding of action content and justifies the use of NL embeddings.

4.3 Multi-Seed Stability

Table 9 reports probe performance across 3 random seeds on the conf≥2 auto-labeled test set. Mean macro-F1 is 0.919 ± 0.011, with all seeds maintaining > 5 pp gap over Repeat-3 (range 0.62–0.84). The low variance confirms training stability.

4.4 Cross-Agent Transfer

We evaluate zero-shot transfer of the SWE-agent–trained probe to 1,247 OpenHands trajectories (18,340 windows). Table 10 shows results. The probe degrades by 9 pp (from 0.80 to 0.71 gold F1), while the LF Oracle (Temporal LM) degrades by 14 pp (from 0.78 to 0.64). The probe's smaller degradation suggests it captures more implementation-agnostic progress signals through embeddings, whereas rule-based LFs are tied to SWE-agent–specific patterns.

Table 8: Non-NLP structural baselines.

Method	Auto Macro-F1	Gold Macro-F1	Features
Structural LR	0.682	0.650	8 numeric
Structural GBDT	0.714	0.680	8 numeric
Repeat-3	0.843	0.630	1 rule
Probe (NL embedding)	**0.906**	**0.800**	1024-dim

Table 9: Multi-seed probe results on conf≥2 auto-labeled test set.

Seed	Macro-F1	Accuracy	F1-Stag	F1-Prog
42	0.906	0.928	0.859	0.952
123	0.927	0.940	0.881	0.973
456	0.923	0.937	0.876	0.970
Mean±Std	0.919±0.011	0.935±0.006	0.872±0.012	0.965±0.011

Per-LF transfer analysis (Table 11) reveals that embedding-based LFs (diversity_drop) transfer well (precision 0.81 → 0.74), while implementation-specific LFs (outcome_bp, tool_misuse) transfer poorly (precision drops to 0.41 and 0.39 respectively), explaining the LF oracle's larger degradation.

4.5 FlyingSquid Comparison

FlyingSquid [Fu et al., 2020] achieves 0.75 gold macro-F1 (+3 pp over MV), identifying 4 LF dependency edges via the triplet method (Table 3). The temporal label model exceeds FlyingSquid by +3 pp. While this margin is moderate, the temporal LM offers two qualitative advantages: (1) interpretable temporal roles for each LF (leading/lagging) vs. FlyingSquid's latent dependency structure, and (2) explicit modeling of sequential structure rather than treating windows as i.i.d. A probe trained on FlyingSquid labels achieves 0.78 gold F1, 2 pp below our temporal LM–trained probe.

4.6 Outcome Back-Propagation Sensitivity

The outcome_bp LF uses trajectory-final information to retrospectively label earlier windows, raising a potential information leakage concern. We quantify this via holdout experiments (Table 12). Completely removing outcome_bp reduces probe gold F1 by 4 pp (from 0.80 to 0.76). Restricting outcome_bp to fire only in the last 3 windows (where 71% of its firings naturally occur) reduces the drop to 2 pp while substantially mitigating leakage. Without outcome_bp, the temporal label model's advantage over MV shrinks from +6 pp to +2 pp, indicating that roughly half of the temporal LM's gain depends on this LF's temporal role.

For practical deployment, we recommend the last-3-only variant as the default configuration, which preserves most of the signal (−2 pp) while confining outcome information to the trajectory's final segment. In offline analysis settings where outcome information is available by definition, the full configuration is appropriate.

4.7 LF Hold-One-Out Ablation

Table 13 shows the marginal contribution of each LF via hold-one-out retraining. Outcome_bp (−3.5 pp) and command_repetition (−2.8 pp) together account for ∼68% of the total LF signal.

Table 10: Cross-agent transfer: SWE-agent → OpenHands.

Method	In-domain (Gold)	Cross-agent	Degradation
LF Oracle (MV)	0.720	0.640	−8.0 pp
LF Oracle (Temporal LM)	0.780	0.640	−14.0 pp
Probe (Temporal LM)	0.800	0.710	−9.0 pp
Repeat-3	0.630	0.580	−5.0 pp

Table 11: Per-LF transfer quality from SWE-agent to OpenHands.

LF	In-domain Prec	Cross-agent Prec	Transfer Quality
command_repetition	0.85	0.62	Partial
outcome_bp	0.78	0.41	Poor
diversity_drop	0.81	0.74	Good
error_pattern	0.76	0.58	Partial
tool_misuse	0.72	0.39	Poor

The sum of individual drops (9.2 pp) exceeds the total gap over MV, indicating substantial inter-LF redundancy—a typical property of data programming systems.

4.8 Ablation Studies

Window Size. Table 14 reports probe performance for window sizes $k \in \{3, 5, 7\}$. $k = 5$ is optimal (gold F1 = 0.80); $k = 3$ degrades by 3.4 pp (insufficient context for temporal patterns to unfold), while $k = 7$ degrades by 1.3 pp (signal dilution within longer windows). The moderate sensitivity to k supports the method's practical robustness.

Architecture. Table 15 compares four architectures. The 2-layer GRU achieves the best tradeoff between capacity and data efficiency. Adding attention yields only +0.6 pp at the cost of additional complexity. Transformer overfits (−2.3 pp) due to its larger parameter count (1.2M vs. 542K) on ∼15K training windows. MLP on mean-pooled features drops 4.9 pp, directly demonstrating the value of sequential modeling.

Encoder Choice. Table 16 compares four embedding models, all with 1024-dimensional output. Performance ranges from 0.78 to 0.80 gold F1 (max spread 2 pp), indicating the method is not critically dependent on encoder choice. Qwen3-Embedding-0.6B achieves the best performance, likely due to stronger code/technical content in its training data.

Data Scaling. Figure 17 shows diminishing returns beyond 25% of training data (∼3,850 windows), which achieves 95% of full performance. Even with 10% of data (1,540 windows), the probe reaches 0.72 gold F1, matching the MV baseline.

SupCon Auxiliary Loss. Adding supervised contrastive loss ($\alpha = 0.5$) improves auto F1 by 0.5 pp and gold F1 by 1 pp, with improved embedding silhouette score (0.42 → 0.52) at 35% additional training time (Table 18). The marginal gain does not justify the added complexity for the default configuration.

Table 12: Outcome back-propagation sensitivity analysis.

Configuration	Auto F1	Gold F1	Δ vs Full
Full (all LFs)	0.906	0.800	—
Hold-out outcome_bp	0.862	0.760	−4.0 pp
outcome_bp last-3 only	0.886	0.780	−2.0 pp
Temporal LM w/o outcome_bp	—	0.740*	−4.0 pp*

*Label quality metric, not probe prediction.

Table 13: LF hold-one-out ablation on auto-labeled test set.

Held-out LF	Macro-F1	Drop (pp)	% of Signal
None (Full)	0.906	—	—
outcome_bp	0.871	−3.5	38%
command_repetition	0.878	−2.8	30%
diversity_drop	0.894	−1.2	13%
error_pattern	0.896	−1.0	11%
tool_misuse	0.899	−0.7	8%

4.9 Error Analysis

On the gold set ($n = 120$ test windows), the probe produces 16 false positives and 12 false negatives (Table 19). Among FPs, 42% involve exploratory actions misclassified as stagnation—the agent tries new approaches that resemble stagnation in embedding space. Among FNs, 55% involve semantically diverse stagnation where the agent executes different but equally ineffective actions, fooling the embedding-based probe. Notably, 63% of probe errors overlap with human annotator disagreement regions, suggesting these are inherently ambiguous cases rather than systematic model failures. Core-region accuracy (non-boundary windows) is 0.94, while boundary-region accuracy is only 0.58.

4.10 Early Termination Simulation

As a downstream application, we simulate stagnation-triggered early termination on 312 held-out trajectories (avg. 48,500 tokens each). At the optimal threshold (0.55), the probe saves 24% of tokens with only 3% success rate drop and 5% false termination rate (Table 20). More aggressive thresholds (0.3) save 42% but incur 12% success rate drop. This demonstrates the probe's calibration: stagnation confidence monotonically predicts actual stagnation probability.

5 Discussion

Temporal Label Model as Interpretable Aggregation. The temporal label model's primary contribution is an interpretable, temporal-aware aggregation mechanism for sequential weak supervision. The +6 pp gain over MV is meaningful but concentrated in two LF pairs, limiting claims of general-purpose temporal label modeling. The advantage over FlyingSquid (+3 pp) is moderate; the temporal LM's main differentiator is explicit interpretability of LF temporal roles rather than raw accuracy.

Table 14: Window size ablation.

k	Auto Macro-F1	Gold Macro-F1	#Windows	Δ vs k=5
3	0.872	0.760	91,348	-3.4 pp
5	**0.906**	**0.800**	54,809	—
7	0.893	0.790	39,149	-1.3 pp

Table 15: Architecture ablation.

Architecture	Auto F1	Gold F1	#Params	Δ vs GRU
MLP (mean-pool)	0.857	0.760	263K	-4.9 pp
Transformer	0.883	0.780	1,247K	-2.3 pp
GRU (default)	**0.906**	**0.800**	542K	—
GRU + Attention	0.912	0.810	609K	$+0.6$ pp

Practical Deployment. The probe's 4 ms inference latency enables real-time monitoring of agent trajectories. Combined with early termination (24% token savings at 3% success cost), the system provides a practical tool for agent orchestration. For deployment, we recommend the last-3-only outcome_bp variant to mitigate information leakage with minimal performance impact (-2 pp).

Representation Geometry. UMAP visualization of GRU hidden states shows class-specific clustering (silhouette = 0.51) with a 22% boundary region where accuracy drops to 0.58. This boundary region corresponds closely to annotator disagreement, suggesting that the probe's errors reflect inherent task ambiguity rather than model limitations.

6 Limitations

Gold Set Quality. $\kappa = 0.68$ is at the borderline of substantial agreement. Ambiguous windows (18%, $\kappa = 0.45$) lower the overall score and may inflate error rates on the gold set.

Outcome BP Dependency. The temporal label model's gain halves (from $+6$ pp to $+2$ pp vs. MV) when outcome_bp is removed, indicating significant dependence on a LF with potential leakage.

Cross-Agent Degradation. While the probe transfers better than LF oracles (-9 pp vs. -14 pp), absolute cross-agent F1 of 0.71 is below acceptable deployment thresholds without further adaptation.

Scope. Both in-domain and cross-agent evaluations involve SE agents (SWE-agent, OpenHands). Generalization to non-SE domains (web browsing, data analysis) is untested.

Temporal LM Generality. Only 2 of 10 possible LF pairs show significant temporal separation, limiting the claim that temporal awareness is broadly beneficial for data programming on sequential data.

Table 16: Encoder ablation.

Encoder	Auto F1	Gold F1	Embed Time	Δ vs Qwen3
Qwen3-Embedding-0.6B	**0.906**	**0.800**	53 min	—
GTE-large	0.894	0.790	45 min	−1.2 pp
BGE-M3	0.891	0.790	48 min	−1.5 pp
E5-large-v2	0.886	0.780	42 min	−2.0 pp

Table 17: Data scaling curve.

Train %	#Windows	Auto F1	Gold F1	Δ vs 100%
10%	1,540	0.811	0.720	−9.5 pp
25%	3,850	0.858	0.760	−4.8 pp
50%	7,700	0.887	0.780	−1.9 pp
100%	15,401	0.906	0.800	—

7 Conclusion

We presented ProgressProbe, a system for window-level progress estimation in LLM agent trajectories. Our temporal label model exploits the leading/lagging dynamics of labeling functions on sequential data, improving label quality by +6 pp over majority vote. A lightweight GRU probe trained on these labels achieves 0.80 gold macro-F1 at 4 ms/window—comparable to LLM-as-judge CoT (−2 pp) at 700× lower latency and ∼850× lower cost. Cross-agent transfer shows the probe captures more implementation-agnostic signals than rule-based LFs, with smaller degradation (−9 pp vs. −14 pp). While the temporal label model's gains concentrate in a few LF pairs and overall novelty is bounded, the combination of temporal-aware weak supervision, efficient inference, and transferable progress signals constitutes a practical and principled approach to monitoring agent behavior at scale.

References

Chen, X., et al. (2024). CDCS and CDSA: Cycle detection for stagnation in LLM agent trajectories. In *Proc. ICSE*, pp. 1–12.

Fu, D. Y., et al. (2020). Fast and three-rious: Speeding up weak supervision with triplet methods. In *Proc. ICML*, pp. 3280–3291.

Lightman, H., et al. (2023). Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Ratner, A., et al. (2017). Snorkel: Rapid training data creation with weak supervision. In *Proc. VLDB*, 11(3):269–282.

Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Proc. NeurIPS*.

Wang, Y., et al. (2024a). SpecRA: Spectral repetition analysis for agent trajectory monitoring. *arXiv preprint*.

Wang, P., et al. (2024b). Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proc. ACL*, pp. 9426–9439.

Table 18: Supervised contrastive loss ablation.

Configuration	Auto F1	Gold F1	Silhouette	Train Time
CE only (default)	0.906	0.800	0.42	1.0×
CE + SupCon α=0.5	0.911	0.810	0.52	1.35×
CE + SupCon α=1.0	0.907	0.805	0.55	1.35×

Table 19: Probe error taxonomy on the gold test set.

Error Type	Count	% of Errors	Dominant Pattern
False Positive	16	57%	Exploration misclassified (42%)
False Negative	12	43%	Semantic diverse stagnation (55%)

Yao, S., et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. In *Proc. NeurIPS*.

Zhang, L., et al. (2024). TrajAD: Anomaly detection in agent trajectories via embedding analysis. *arXiv preprint*.

Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Proc. NeurIPS*.

Zhou, A., et al. (2023). Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*.

Qwen Team. (2025). Qwen3-Embedding: A multilingual embedding model. Technical report, Alibaba Group.

Table 20: Early termination tradeoff on 312 held-out trajectories.

Threshold	Tokens Saved	Success Drop	False Term. Rate
0.3	42%	12%	0.18
0.5	28%	5%	0.08
0.55 (optimal)	24%	3%	0.05
0.7	15%	2%	0.03